

DS n° 1**CORRECTION****EXERCICE 1 : CONSTRUCTIONS DE LISTES**

1) On récupère ([2, 8], [3, 5, 1])

```
1)
1 def separation(L):
2     tA,tB = [] ,[]    # on initialise 2 tableau vide
3     for val in L :    # on parcourt L en valeur
4         if val %2 == 0:    # si val est pair
5             tA.append(val)    # on le place dans tA
6         else:
7             tB.append(val)    # sinon on le place dans tB
8     return tA,tB    # on retourne le couple ta,tB
```

```
2)
1 def signe(L):
2     tP,tN = [] ,[]
3     for val in L :
4         if val >=0 :
5             tP.append(val)
6         else:
7             tN.append(val)
8     return tP,tN
```

EXERCICE 2 : RECHERCHE DANS UN TABLEAU

```
1)
1 def recherche(tab,x):
2     for val in tab :
3         if val == x :
4             return True
5     return False
```

2) On fait au pire n comparaisons si l'élément n'est pas dans le tableau.

```
2)
1 def recherchepos(tab,x):
2     for i in range(len(tab)) :
3         if tab[i] == x :
4             return i
5     return None
```

```
4)
1 def ttepos(tab,x):
2     L = []
3     for i in range(len(tab)) :
4         if L[i] == x :
5             L.append(i)
6     return L
```

```
5)
1 def compte(tab,x):
2     c = 0
3     for val in tab :
4         if val == x :
5             c += 1
6     return c
```

```

6)
1 def egal(tab1, tab2):
2     for k in range(len(tab1)):
3         if tab1[k] != tab2[k]:
4             return False
5     return True

```

```

7)
1 def tabegal(tab1, tab2):
2     T = []
3     for k in range(len(tab1)):
4         T.append(tab1[k] == tab2[k])
5     return T

```

```

8)
1 def nbegal(tab1, tab2):
2     n = 0
3     for k in range(len(tab1)):
4         if tab1[k] == tab2[k]:
5             n = n + 1
6     return n

```

9) On fait une comparaison par passage en boucle, donc n comparaisons au total.

```

10)
1 def nbglobalegal(tab1, tab2):
2     n = 0
3     for val in tab1:
4         for res in tab2:
5             if val == res:
6                 n = n + 1
7     return n

```

11) Ici on a une double boucle, on passe n fois dans la première, p dans la seconde, et on fait une comparaison, donc np comparaisons en tout.

EXERCICE 3 : ETUDE DE HOULE

1) On a $H_1 \simeq 9m$, $H_2 \simeq 9m$ et $H_3 \simeq 6m$ et $T_1 \simeq 12s$ et $T_2 \simeq 13s$.

2) `def moyenne(liste_niveaux):`
`return sum(liste_niveaux)/len(liste_niveaux)`

3) `1 def construction_successeurs(liste_niveaux):`
`2     n = len(liste_niveaux)`
`3     successeurs = []`
`4     m = moyenne(liste_niveaux)`
`5     for i in range(n-1):`
`6         if liste_niveaux[i] > moyenne and liste_niveaux[i+1] < moyenne :`
`7             successeurs.append(i+1)`
`8     return successeurs`

4) `def decompose_vagues(liste_niveaux):`
`successeurs = construction_successeurs(liste_niveaux)`
`vagues = []`
`for i in range(len(successeurs)-1):`
`Zi = successeurs[i]`
`Zip1 = successeurs[i+1]`
`vagues.append(liste_niveaux[Zi:Zip1])`
`return vagues`

```
5) def proprietes(liste_niveaux):  
    vagues = decompose_vagues(liste_niveaux)  
    dt = 0.5  
    P = []  
    for vague in vagues :  
        Hi = max(vague) - min(vague)  
        Ti = len(vague)*dt  
        P.append([Hi,Ti])  
    return P
```